LAB 15 - PROMPTS

Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab15-trading-costs/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Read the cost model (K7)

Prompt - copy into your AI coding assistant
Explain CostModel.trade_cost in labs_fullcode/common/backtest.py with a worked example: a USD 95,000 buy and the matching sell of SPY at 676.64 under the alpaca_retail preset. Show each component in dollars. Do not change the file.

Open in Claude | Open in ChatGPT

Check: A worked example where spread and slippage dominate and commission is zero.

Step 2: Build the engine helpers (A7)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/backtest.py. run_backtest(bars, name, costs=None, execution='next_open', **params) = common.backtest.backtest(bars, pipeline.signals.target_position(name, bars, **params), costs, execution) - the single entry point later stages use. break_even_bps(bars, signal, lo=0, hi=1000): bisection (30 steps) on a one-way cost expressed as CostModel(slippage_bps=x) until net CAGR = 0 (return 0 if gross CAGR <= 0). parse_custom(['spread_bps=3', 'slippage_bps=3']) -> CostModel('Custom route', ...). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: break_even_bps on dip_buy returns about 19.5.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the scenario report (A7, A8)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/backtest.py add run(strategies=None, symbol='SPY', custom=None) for `python -m pipeline backtest --strategies --symbol --custom`: default strategies dip_buy and trend_filter; scenarios from config costs.scenarios plus the custom one; for each strategy and scenario print gross CAGR, net CAGR, cost drag, net Sharpe, executions and dollars of commission, spread, slippage, reg fees and total (from BacktestResult.cost_summary()); print each strategy's break-even bps per side next to its number of round trips and average gross trade in bps; print next-bar OPEN versus next-bar CLOSE CAGR and Sharpe with alpaca_retail costs; save _outputs/Lab15_cost_scenarios.csv and chart Lab15_costs.png (equity by scenario for the first strategy with the gross curve dashed, and a stacked bar of where the costs go). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline backtest --help shows --strategies, --symbol and --custom.

Step 4: Run the cost scenarios (A7, A8)

Do this (no prompt needed)
Run: python -m pipeline backtest --strategies dip_buy trend_filter (same as: python3 Lab15-trading-costs/backtest_stage.py).

Check: dip_buy: gross CAGR 5.09%; net 4.65% (Alpaca), 4.21% (per-share), 0.47% (high-cost, USD 33,560 of costs); 203 executions. trend_filter: gross 8.80%; net 8.70% / 8.59% / 7.68%; 47 executions.

Step 5: Analyse the cost breakdown (K7)

Do this (no prompt needed)
For the Alpaca scenario, which component is largest even though commission is zero?

Check: dip_buy with Alpaca costs: commission USD 0, spread 1,127, slippage 2,254, reg fees 317 - total USD 3,698; slippage and spread scale with turnover.

Step 6: Interpret break-even and execution timing (A8, K7)

Do this (no prompt needed)
Compare each break-even cost with the average trade, then read the execution-timing lines.

Check: Break-even: dip_buy 19.5 bps per side (102 round trips, average gross trade 41.7 bps) versus trend_filter 142.3 bps (24 round trips, 347.3 bps). Timing: dip_buy 4.65% at the next OPEN but 7.22% at the next CLOSE - the fill assumption alone moves the result; trend_filter 8.70% vs 8.66%.

Step 7: Configure your own execution route (A7)

Do this (no prompt needed)
Run: python -m pipeline backtest --custom spread_bps=3 slippage_bps=3 and record the new net CAGR for both strategies.

Check: Custom route: dip_buy net 3.89% (USD 9,833 of costs); trend_filter 8.51% (USD 2,445).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Run it

Step	Command / file
Run Lab 15	python3 Lab15-trading-costs/backtest_stage.py (same as: python -m pipeline backtest)
The complete code	pipeline/backtest.py (reference copy: Lab15-trading-costs/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.